Unite2:Assignment6.py:

```
:"""
======================================================================
 OPTIMAL SELECTION OF ITEMS WITHIN A WEIGHT CONSTRAINT
 (The classic 0/1 Knapsack Problem)
 Approaches included: Brute Force (for comparison), Top-Down
                      (Memoization), and Bottom-Up (Tabulation).
======================================================================
IN PLAIN ENGLISH:
  You're packing a bag with a weight limit. Each item has a weight
  and a value. You want the most valuable combination of items that
  still fits in the bag. You can't take a fraction of an item \u2014
  either it goes in the bag, or it doesn't. That's why this is
  called the "0/1" Knapsack Problem (0 = leave it, 1 = take it).
======================================================================
"""

import timeit


# ======================================================================
# 1. BRUTE FORCE  (for comparison only \u2014 exponential time)
#    Plain English: try every possible combination of items \u2014 for
#    each item, either include it or don't, and keep the best result.
# ======================================================================
def knapsack_brute_force(weights, values, capacity, n=None):
    """Naive recursive solution. Time complexity: O(2^n)."""
    if n is None:
        n = len(weights)
    if n == 0 or capacity == 0:
        return 0
    if weights[n - 1] > capacity:
        # This item can't fit no matter what \u2014 skip it.
        return knapsack_brute_force(weights, values, capacity, n - 1)
    include = values[n - 1] + knapsack_brute_force(weights, values,
capacity - weights[n - 1], n - 1)
    exclude = knapsack_brute_force(weights, values, capacity, n - 1)
    return max(include, exclude)
```

```
# ============================================================================
# 2. TOP-DOWN  (memoized recursion)
#    Plain English: same include/exclude recursion as brute force, but
#    remembers the answer for every (item, remaining capacity) pair it
#    has already solved.
# ============================================================================
def knapsack_top_down(weights, values, capacity):
    """Recursive solution with a cache. Time complexity: O(n *
capacity)."""
    n = len(weights)
    memo = {}

    def helper(i, remaining_capacity):
        if i == 0 or remaining_capacity == 0:
            return 0
        if (i, remaining_capacity) in memo:
            return memo[(i, remaining_capacity)]

        if weights[i - 1] > remaining_capacity:
            result = helper(i - 1, remaining_capacity)
        else:
            include = values[i - 1] + helper(i - 1, remaining_capacity -
weights[i - 1])
            exclude = helper(i - 1, remaining_capacity)
            result = max(include, exclude)

        memo[(i, remaining_capacity)] = result
        return result

    return helper(n, capacity)


# ============================================================================
# 3. BOTTOM-UP  (tabulation) \u2014 the main, recommended solution
#    Plain English: build a grid where rows are items and columns are
#    "capacity used so far". Fill it from the top-left, one cell at a
#    time, using only cells already filled.
# ============================================================================
def build_knapsack_table(weights, values, capacity):
    """Fills an (n+1) x (capacity+1) grid where dp[i][w] is the best
```

```
    value achievable using only the first i items, with a weight
    limit of w."""
    n = len(weights)
    dp = [[0] * (capacity + 1) for _ in range(n + 1)]

    for i in range(1, n + 1):
        for w in range(capacity + 1):
            if weights[i - 1] > w:
                dp[i][w] = dp[i - 1][w]                    # can't fit ->
skip it
            else:
                skip = dp[i - 1][w]
                take = values[i - 1] + dp[i - 1][w - weights[i - 1]]
                dp[i][w] = max(skip, take)                 # best of
skipping or taking
    return dp


def reconstruct_selected_items(dp, weights, values, capacity):
    """Walks BACKWARDS through the filled table to recover WHICH items
    were actually chosen, not just the best total value."""
    n = len(weights)
    w = capacity
    selected = []
    for i in range(n, 0, -1):
        if dp[i][w] != dp[i - 1][w]:
            selected.append(i - 1)      # item index (0-based) was
included
            w -= weights[i - 1]
    selected.reverse()
    return selected


def knapsack_bottom_up(weights, values, capacity):
    """Main entry point.
    Returns: (max_value, selected_item_indices, dp_table)"""
    dp = build_knapsack_table(weights, values, capacity)
    n = len(weights)
    max_value = dp[n][capacity]
    selected = reconstruct_selected_items(dp, weights, values, capacity)
```

```python
    return max_value, selected, dp


# ============================================================================
# 4. DEMO / DRIVER CODE
# ============================================================================
if __name__ == "__main__":

    weights = [2, 3, 4, 5]
    values = [3, 4, 5, 6]
    capacity = 5

    print("Items (weight, value):", list(zip(weights, values)))
    print(f"Knapsack capacity: {capacity}\n")

    print(f"[1] Brute force : {knapsack_brute_force(weights, values,
capacity)}")
    print(f"[2] Top-down    : {knapsack_top_down(weights, values,
capacity)}")

    max_value, selected, dp = knapsack_bottom_up(weights, values,
capacity)
    print(f"[3] Bottom-up   : {max_value}")
    print(f"    Selected items (0-indexed): {selected}")
    for idx in selected:
        print(f"      item {idx}: weight={weights[idx]},
value={values[idx]}")

    # ---- cross-check: brute force, top-down, and bottom-up all agree
----
    assert knapsack_brute_force(weights, values, capacity) == max_value
    assert knapsack_top_down(weights, values, capacity) == max_value
    print("\n[check] All three approaches agree on the maximum value.")

    # ---- timing comparison on a moderately large input
-------------------
    import random
    random.seed(42)
    n_big = 22
    big_weights = [random.randint(1, 15) for _ in range(n_big)]
```

```
    big_values = [random.randint(1, 20) for _ in range(n_big)]
    big_capacity = 50

    t_brute = timeit.timeit(lambda: knapsack_brute_force(big_weights,
big_values, big_capacity), number=1)
    t_bottom_up = timeit.timeit(lambda: knapsack_bottom_up(big_weights,
big_values, big_capacity), number=1)
    print(f"\nTiming with {n_big} items, capacity {big_capacity}:")
    print(f"  brute force : {t_brute:.5f} sec")
    print(f"  bottom-up   : {t_bottom_up:.8f} sec  (thousands of times
faster)")



    ---------output------------
    Items (weight, value): [(2, 3), (3, 4), (4, 5), (5, 6)]
Knapsack capacity: 5

[1] Brute force : 7
[2] Top-down    : 7
[3] Bottom-up   : 7
    Selected items (0-indexed): [0, 1]
      item 0: weight=2, value=3
      item 1: weight=3, value=4

[check] All three approaches agree on the maximum value.

Timing with 22 items, capacity 50:
  brute force : 0.33194 sec
  bottom-up   : 0.00030412 sec  (thousands of times faster)
```